



SOFTWARE ENGINEERING

C03001

CHAPTER 8 — SOFTWARE IMPLEMENTATION

Anh Nguyen-Duc
Tho Quan-Thanh



Adapted from <https://iansommerville.com/software-engineering-book/slides/>

WEEK 8

TOPICS COVERED

- ✓ Implementation meaning
- ✓ Coding style & standards
- ✓ Code with correctness justification
- ✓ Code smell and some good practices
- ✓ Code inspection
- ✓ Exercise

IMPLEMENTATION

smallest part that will be separately maintained

Implementation = Unit Implementation + Integration

put them all together

IMPLEMENT CODE

One way to ...

- ✓ 1. Plan the structure and residual design for your code
- ✓ 2. Self-inspect your design and/or structure
- ✓ 3. Type your code
- ✓ 4. Self-inspect your code
- ✓ 5. Compile your code
- ✓ 6. Test your code

GENERAL PRINCIPLES IN PROGRAMMING PRACTICE

- ✓ 1. Try to re-use first
- ✓ 2. Enforce intentions
 - If your code is intended to be used in particular ways only, write it so that the code cannot be used in any other way.
 - If a member is not intended to be used by other functions, enforce this by making it private or protected etc.
 - Use qualifiers such as final and abstract etc. to enforce intentions

“THINK GLOBALLY, PROGRAM LOCALLY”

- ✓ Make all members
 - as local as possible
 - as invisible as possible
 - attributes private:
 - access them through more public accessor functions if required.
 - (Making attributes protected gives objects of subclasses access to members of their base classes -- not usually what you want)

EXCEPTIONS HANDLING

*“If you must choice between throwing an exception and continuing the computation, continue if you can”
(Cay Horstmann)*

- ✓ Catch only those exceptions that you know how to handle
- ✓ Be reasonable about exceptions callers must handle
- ✓ Don't substitute the use of exceptions for issue that should be the subject of testing

NAMING CONVENTIONS

- ✓ Use concatenated words
 - e.g., cylinderLength
- ✓ Begin class names with capitals
- ✓ Variable names begin with lowercase
- ✓ Constants with capitals
 - as in MAX_N or use static final
- ✓ Data members of classes with an underscore
 - as in _timeOfDay
- ✓ Use get..., set..., and is... for accessor methods
- ✓ Additional getters and setters of collections
- ✓ And/or distinguish between instance variables, local variables and parameters

DOCUMENTING METHODS

- ✓ what the method does
- ✓ why it does so
- ✓ what parameters it must be passed (use @param tag)
- ✓ exceptions it throws (use @exception tag)
- ✓ reason for choice of visibility
- ✓ known bugs
- ✓ test description, describing whether the method has been tested, and the location of its test script
- ✓ history of changes if you are not using a sub-version system
- ✓ example of how the method works
- ✓ pre- and post-conditions
- ✓ special documentation on threaded and synchronized methods

```

/* Class Name      : EncounterCast
* Version information : Version 0.1
* Date            : 6/19/1999
* Copyright Notice  : see below
* Edit history:
* 11 Feb /** Facade class/object for the EncounterCharacters package. Used to
* 8 Feb  * reference all characters of the Encounter game.
* 08 Jan * <p> Design: SDD 3.1 Module decomposition
*/      * <br> SDD 5.1.2 Interface to the EncounterCharacters package
*
/*      * <p>Design issues:<ul>
Copyrig * <li> SDD 5.1.2.4 method engagePlayerWithForeignCharacter was
*      * not implemented, since engagements are handled more directly
This pro * from the Engaging state object.
"Softwa * </u> /** Gets encounterCast, the one and only instance of EncounterCast.
by Eric  *      * <p> Requirement: SDD 5.1.2
* @a      *
* @va     * @return    The EncounterCast singleton.
*/        */
public    public static EncounterCast getEncounterCast()
{          { return encounterCastS; }

/** Name for human player */
private static final String MAIN_PLAYER_NAME = "Elena";

```

DOCUMENTING ATTRIBUTES

- ✓ Description -- what it's used for
- ✓ All applicable invariants
 - quantitative facts about the attribute,
 - such as " $1 < \text{_age} < 130$ "
 - or " $36 < \text{_length} * \text{_width} < 193$ ".

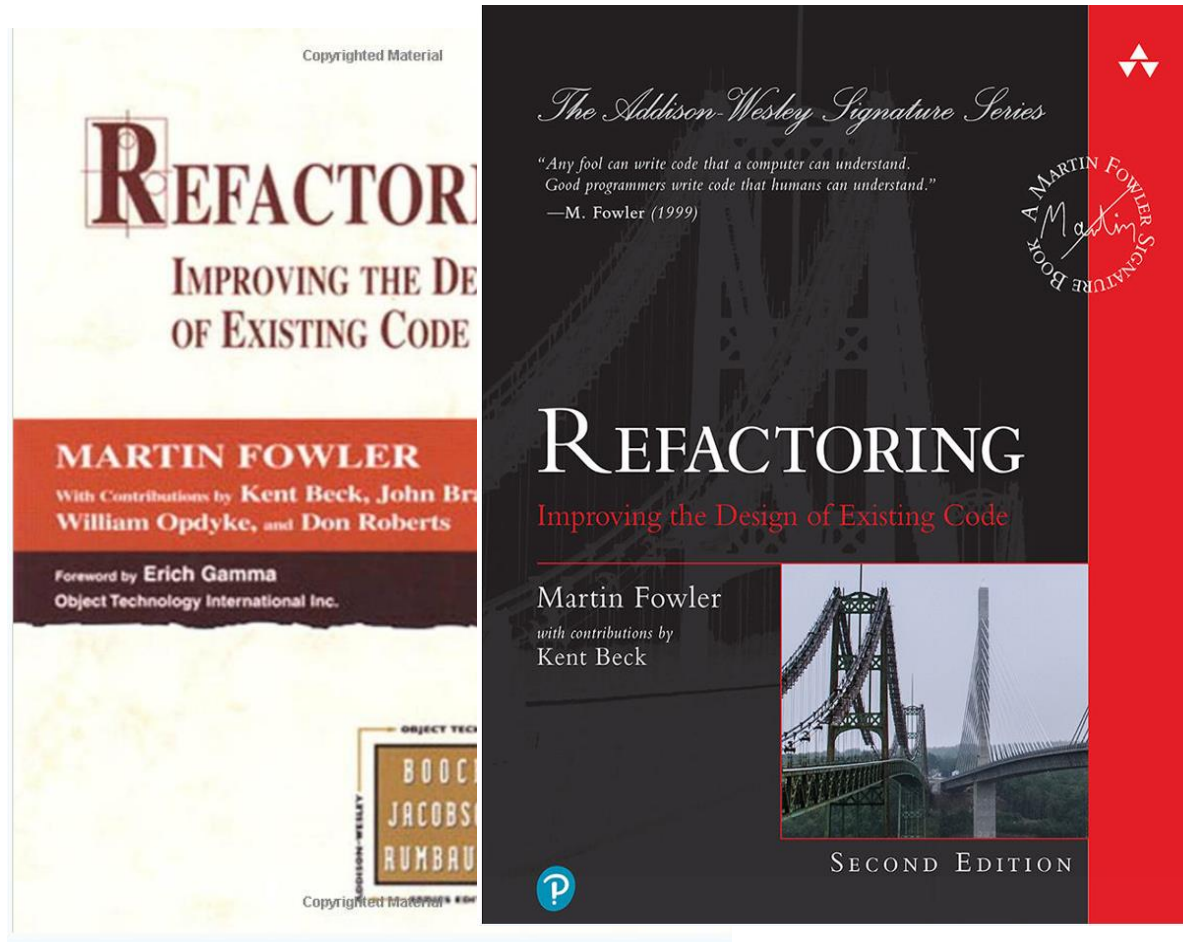
CONSTANTS

- ✓ Before designating a final variable, be sure that it is, indeed, final. You're going to want to change "final" quantities in most cases. Consider using method instead.
- ✓ Ex:
 - instead of ...
 - `protected static final MAX_CHARS_IN_NAME;`
 - consider using ...
 - `protected final static int getMaxCharsInName() { return 20; }`

INITIALIZING ATTRIBUTES

- ✓ Attributes should be always be initialized, think of
 - `private float _balance = 0;`
- ✓ Attribute may be an object of another class, as in
 - `private Customer _customer;`
- ✓ Traditionally done using the constructor, as in
 - `private Customer _customer = new Customer( "Edward", "Jones" );`
- ✓ Problem is maintainability. When new attributes added to Customer, all have to be updated. Also accessing persistent storage unnecessarily.

RELEVANT BOOKS



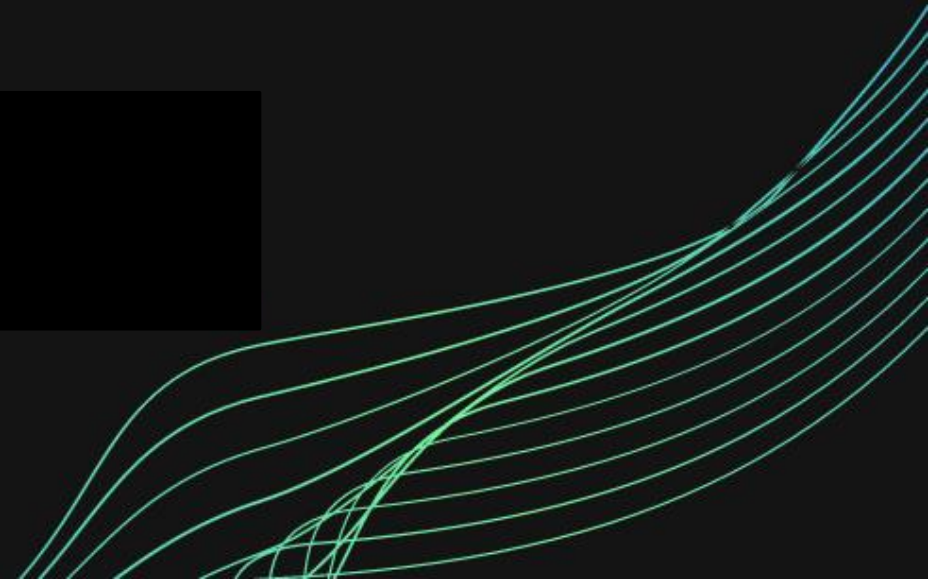
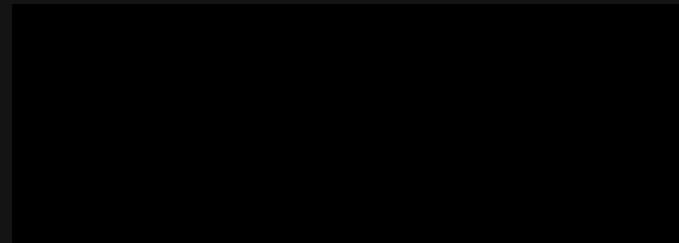
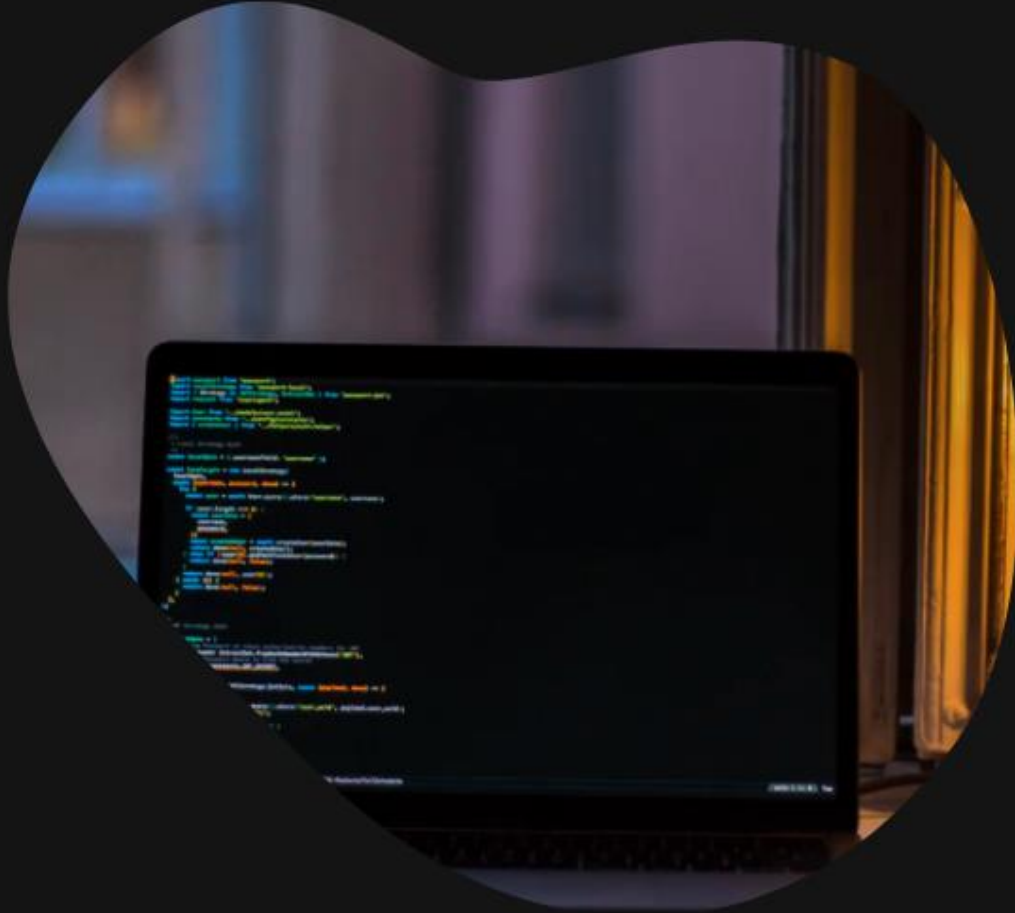
New edition

BAD SMELLS IN CODE (CODE SMELLS)

- ✓ Code smells are any violation of fundamental design principles that decrease the overall quality of the code.
- ✓ Not bugs or errors
- ✓ Can certainly add to the chance of bugs and failures down the line.

Code Smells

That Are Found The Most





Bloaters

Bloaters are code, methods and classes that have increased to such gargantuan proportions that they are hard to work with. Usually these smells do not crop up right away, rather they accumulate over time as the program evolves (and especially when nobody makes an effort to eradicate them).

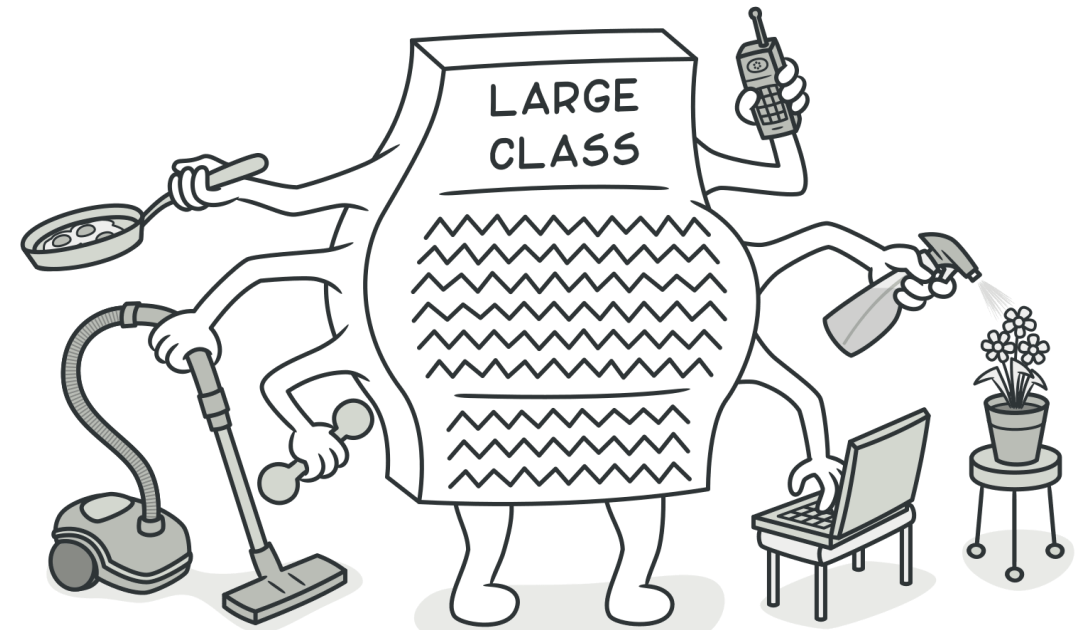
§ Long Method

§ Large Class

§ Primitive Obsession

§ Long Parameter List

§ Data Clumps





Object-Orientation Abusers

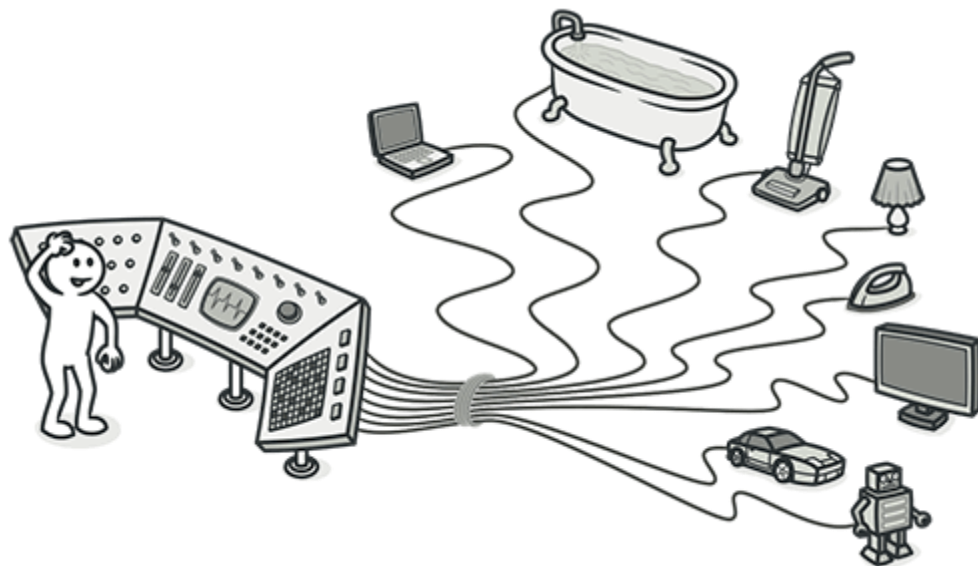
All these smells are incomplete or incorrect application of object-oriented programming principles.

§ Alternative Classes with Different Interfaces

§ Refused Bequest

§ Temporary Field

§ Switch Statements





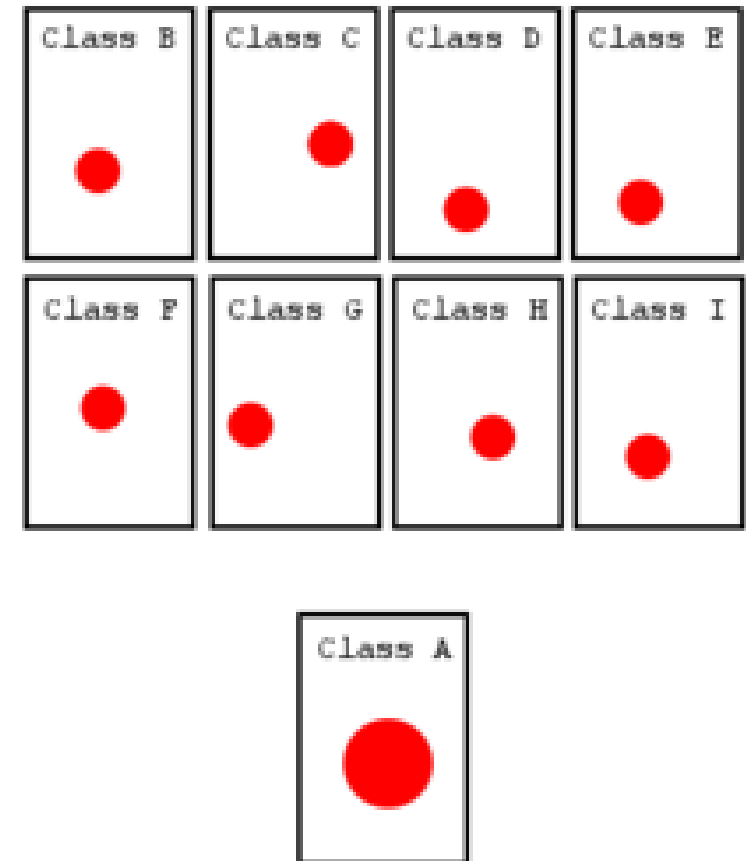
Change Preventers

These smells mean that if you need to change something in one place in your code, you have to make many changes in other places too. Program development becomes much more complicated and expensive as a result.

§ Divergent Change

§ Parallel Inheritance Hierarchies

§ Shotgun Surgery



Dispensables

A dispensable is something pointless and unneeded whose absence would make the code cleaner, more efficient and easier to understand.

§ Comments

§ Duplicate Code

§ Data Class

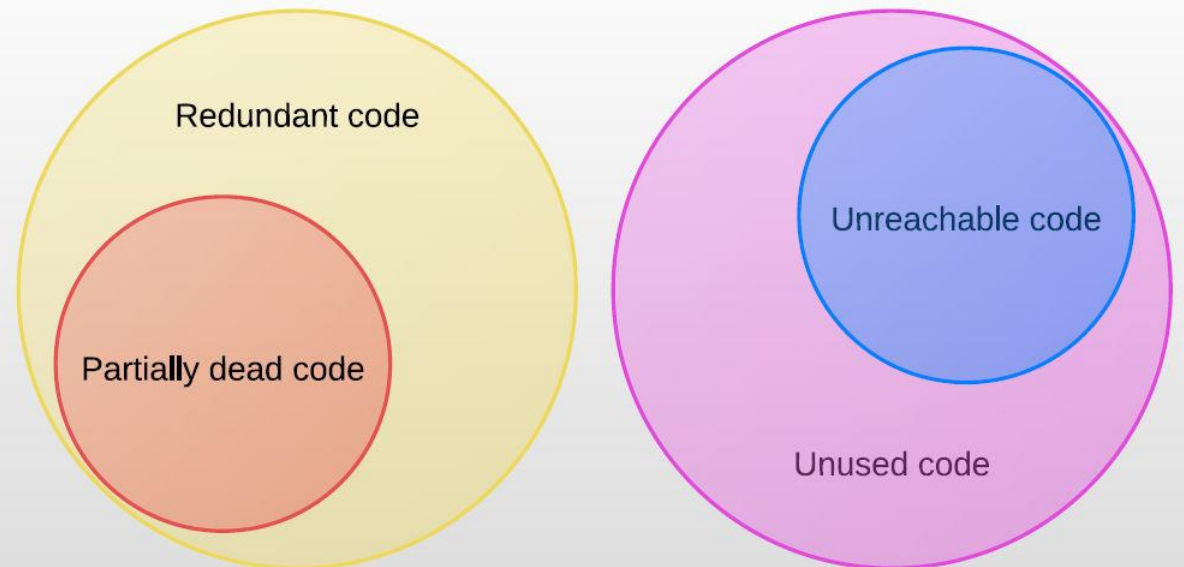
§ Dead Code

§ Lazy Class

§ Speculative Generality



Source code





Couplers

All the smells in this group contribute to excessive coupling between classes or show what happens if coupling is replaced by excessive delegation.

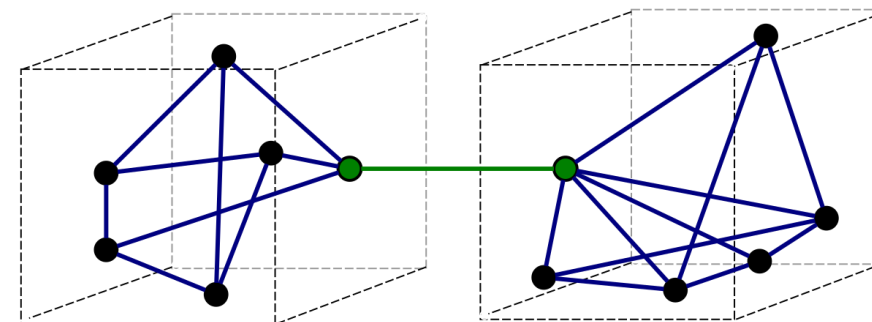
§ Feature Envy

§ Inappropriate Intimacy

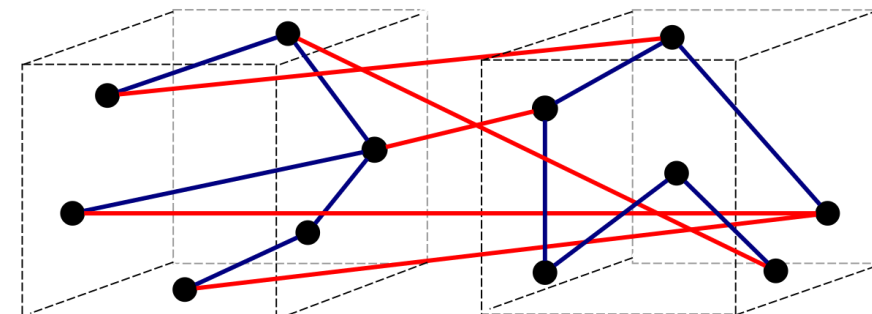
§ Incomplete Library Class

§ Message Chains

§ Middle Man



a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

COD REFACTORING GOALS AND PROPERTIES

- ✓ Change the internal structure without changing external behavior
- ✓ Eliminate code smells for
 - Readability
 - Consistency
 - Maintainability
- ✓ Properties
 - Preserve correctness
 - One step at a time
 - Frequent testing

WHY DO WE NEED INSPECTION?

- ✓ Many software artifacts cannot be verified by running tests, e.g.,
 - Requirement specification
 - Design
 - Pseudocode
 - User manual
- ✓ Inspection reduces defect rates
- ✓ Inspection complements testing
- ✓ Inspection identifies code smells
- ✓ Inspection provides additional benefits

INSPECTION FINDS TYPES OF DEFECTS DIFFERENT FROM TESTING

Number of different types of defects detected by testing vs. inspection

Some defect types	Testing	Inspection
Uninitialized variables	1	4
Illegal behavior, e.g., division by zero	49	20
Incorrectly formulated branch conditions	2	13
Missing branches, including both conditionals and their embedded statements	4	10

TESTING VS. INSPECTION

	Testing	Inspection
Pros	• Can, at least partly, be automated • Can consistently repeat several actions • Fast and high volume	• Can be used on all types of documents, not just code • Can see the complete picture, not only a spot check • Can use all information in the team • Can be innovative and inductive
Cons	• Is only a spot check • Can only be used for code • May need several stubs and drivers	• Is difficult to use on complex, dynamic situations • Unreliable (people can get tired) • Slow and low volume

CODE REVIEW AT GOOGLE

- ✓ “All code that gets submitted needs to be **reviewed by at least one other person**, and either the code writer or the reviewer needs to have readability in that language”
- ✓ “Most people use **Mondrian** to do code reviews, and obviously, **we spend a good chunk of our time reviewing code**”
--Amanda Camp, Software Engineer, Google

INSPECT CODE 1 OF 5: CLASSES OVERALL

One way to ...

- ✓ C1. Is its (the class') name appropriate?
- ✓ C2. Could it be abstract (to be used only as a base)?
- ✓ C3. Does its header describe its purpose?
- ✓ C4. Does its header reference the requirements and/or design element to which it corresponds?
- ✓ C5. Does it state the package to which it belongs?
- ✓ C6. Is it as private as it can be?
- ✓ C7. Should it be final (Java)
- ✓ C8. Have the documentation standards been applied?

INSPECT CODE 2 OF 5 : ATTRIBUTES

One way to ...

- ✓ A1. Is it (the attribute) necessary?
- ✓ A2. Could it be static?
- ✓ A3. Should it be final?
- ✓ A4. Are the naming conventions properly applied?
- ✓ A5. Is it as private as possible?
- ✓ A6. Are the attributes as independent as possible?
- ✓ A7. Is there a comprehensive initialization strategy?

INSPECT CODE 3 OF 5 : CONSTRUCTORS

One way to ...

- ✓ CO1. Is it (the constructor) necessary?
- ✓ CO2. Does it leverage existing constructors?
- ✓ CO3. Does it initialize of all the attributes?
- ✓ CO4. Is it as private as possible?
- ✓ CO5. Does it execute the inherited constructor(s) where necessary?

INSPECT CODE 4 OF 5: METHOD HEADERS

One way to ...

- ✓ MH1. Is the method appropriately named?
- ✓ MH2. Is it as private as possible?
- ✓ MH3. Could it be static?
- ✓ MH4. Should it be final?
- ✓ MH5. Does the header describe method's purpose?
- ✓ MH6. Does the method header reference the requirements and/or design section that it satisfies?
- ✓ MH7. Does it state all necessary invariants?
- ✓ MH8. Does it state all pre-conditions?
- ✓ MH9. Does it state all post-conditions?
- ✓ MH10. Does it apply documentation standards?
- ✓ MH11. Are the parameter types restricted?

INSPECT CODE 5 OF 5: METHOD BODIES

One way to ...

- ✓ MB1. Is the algorithm consistent with the detailed design pseudocode and/or flowchart?
- ✓ MB2. Does the code assume no more than the stated preconditions?
- ✓ MB3. Does the code produce every one of the postconditions?
- ✓ MB4. Does the code respect the required invariant?
- ✓ MB5. Does every loop terminate?
- ✓ MB6. Are required notational standards observed?
- ✓ MB7. Has every line been thoroughly checked?
- ✓ MB8. Are all braces balanced?
- ✓ MB9. Are illegal parameters considered?
- ✓ MB10. Does the code return the correct type?
- ✓ MB11. Is the code thoroughly commented?